Horizon spawn and screen spawn

Technical note for the landscape runner demo. Written for a Flutter and Flame developer. Constants match the playable demo: `NEAR = 260`, `VIEW_Z = 2300`, `PLAYER_Z = 90`, `SPEED = 520`.

Shared world

Both views play one run. `z` is the distance ahead of the camera. Larger `z` is farther up the road. The avatar stands at `PLAYER_Z = 90`. Items are spawned out to `VIEW_Z = 2300` and approach by decreasing `z` at `SPEED = 520` world units per second. A lane is a fixed road coordinate `nx` (`-0.64`, `0`, `0.64`), not a pixel offset.

One scale value, `s`, drives screen x, screen y, and sprite scale. The sprite is anchored at its feet. In Flame that is `Anchor.bottomCenter`. `SpriteComponent` defaults to `Anchor.topLeft`, and a growing sprite then slides off the lane it is supposed to sit in.

`horizonY` is 36% of the way down the panel. `groundY` is the bottom of the panel. `NEAR` is the perspective constant. A smaller `NEAR` pulls the vanishing point closer and makes the near road wider.

The two formulas

```
function project(nx, z) {
  let s;
  if (viewMode === "screen") {
    const farS = NEAR / (NEAR + VIEW_Z);
    const pinS = NEAR / (NEAR + PLAYER_Z);
    const t = clamp((VIEW_Z - z) / (VIEW_Z - PLAYER_Z), 0, 1.08);
    s = farS + (pinS - farS) * t;
  } else {
    s = NEAR / (NEAR + Math.max(0.001, z));
  }
  return {
    x: width * 0.5 + nx * width * 0.34 * s,
    y: horizonY + (groundY - horizonY) * s,
    s: Math.max(0.001, s),
  };
}
```

Screen x, screen y, and scale all read `s`. The modes differ only in how `z` becomes `s`.

Horizon spawn

```
s = NEAR / (NEAR + z)
x = centerX + nx * roadHalfWidth * s
y = horizonY + (groundY - horizonY) * s
scale = s
```

This is a pinhole projection of a ground plane, with the camera looking along `z`. With the camera eating distance at a constant speed `v`:

```
ds/dz = -NEAR / (NEAR + z)²
|ds/dt| = v * NEAR / (NEAR + z)²
```

`|ds/dt|` is small when `z` is large and large when `z` is small. The same 100 units of road is a few pixels at the horizon and a long stretch beside the avatar. Because scale uses that same `s`, a coin's width stays matched to the lane width for the whole trip.

In Flame, store a world distance on the component, move that distance, then project. Road dashes, trees, lamps, and buildings call the same function with the same camera. Nearer sprites get a higher priority so they paint on top. The sky stays a normal backdrop. It does not share this distance. Flame's `ParallaxComponent` scrolls sideways on the camera and is the wrong tool for a road seen from behind the player.

```
distance -= runSpeed * dt;
final s = near / (near + distance);
scale = Vector2.all(s);
position = Vector2(
  centerX + laneOffset * s,
  horizonY + (groundY - horizonY) * s,
);
anchor = Anchor.bottomCenter;
priority = -distance.round();
```

Use `near = 260` and `runSpeed = 520` to match the demo. An older `flame_depth.dart` sketch in the same folder uses `near = 780` and a faster run speed from an earlier camera. Those values do not match this demo.

Screen spawn

The road on this view is drawn once with the horizon formula and the camera held at `z = 0`. Dashes and roadside props do not scroll. Only coins, suns, and bats use the other branch.

That branch still feeds `s` into x, y, and scale, but `s` is a straight blend from the horizon scale to the avatar scale:

```
t = (VIEW_Z - z) / (VIEW_Z - PLAYER_Z)     // 0 at the horizon, 1 at the avatar
s = s(VIEW_Z) + (s(PLAYER_Z) - s(VIEW_Z)) * t
```

`t` is clamped so an item appears at the horizon scale and stops just past the pin. `ds/dz` is constant, so equal gaps in `z` are equal gaps in pixels. A coin pair stays evenly spaced from the horizon to the avatar, instead of compressing into the distance and opening up at the end.

Hit testing is the same world test on both views. A bat or a pickup counts when its `z` is within a few dozen units of `PLAYER_Z` and it shares the avatar's lane. Because `t = 1` is defined as `z = PLAYER_Z`, the sprite is on the pin at the moment the hit counts. An earlier version of screen spawn ran `t` all the way to the bottom edge of the panel, so the score changed after the item had already passed the avatar.

The original screen-step bug

The runner behavior this demo is comparing against steps a screen progress value and copies it into both position and scale. The road used that same progress, so dashes and trees stepped forward in even slices and spread apart as they arrived.

```
screenT += dt / tripSeconds;
scale = Vector2.all(screenT);
position = Vector2(
  centerX + laneOffset * screenT,
  horizonY + (groundY - horizonY) * screenT,
);
```

`screenT` runs from 0 to 1 in a fixed number of seconds. Pixel speed is flat, so the end of the trip looks slow and sprites pile up at the bottom. The top view of the demo keeps the linear item motion, aims it at the pin instead of the bottom edge, and leaves the road on the perspective formula so only the items show the difference.

Worked numbers

With `NEAR = 260`, the horizon formula at the two ends of the trip is:

Distance	Horizon s	Screen s
Horizon, `z = 2300`	`260 / 2560 ≈ 0.10`	`0.10` (the blend starts here)
Midway, `z = 1195`	`260 / 1455 ≈ 0.18`	`0.42`
Avatar, `z = 90`	`260 / 350 ≈ 0.74`	`0.74` (the blend ends here)

The two modes agree at the horizon and at the avatar. They disagree in the middle. Halfway along the trip, the screen-spawn item is already more than twice as large, and more than twice as far down the panel, as the horizon-spawn item.

At `v = 520`, horizon `|ds/dt|` is about `0.021` per second at `z = 2300` and about `1.10` per second at `z = 90`. Screen `|ds/dt|` is about `0.151` per second for the whole trip:

```
|ds/dt| = 520 * (0.743 - 0.102) / (2300 - 90) ≈ 0.151
```

That constant rate is about 7 times the horizon rate at the horizon, and about 7 times slower than the horizon rate at the pin. The approach feels even. The horizon approach feels slow in the distance and then opens up beside the avatar.

What stays the same

- One spawn list, one lane, one score, one blink-and-slow on a bat hit.
- Collision is a `z` window around `PLAYER_Z` plus a lane test. It is not a pixel test against the bottom of the screen.
- A gate always leaves one lane open. Coin runs are two coins in one lane. Bats never share a lane with that beat's pickup.